



Pertemuan 11

▼ Penggunaan `useEffect`



<https://www.linkedin.com/pulse/understanding-power-useeffect-reactjs-comprehensive-guide-bellal-/>

Jika `useState` adalah hook di React yang digunakan untuk menyimpan data (state) dalam sebuah komponen, maka `useEffect` adalah hook yang digunakan untuk menjalankan kode setelah komponen selesai di-render.

`useEffect` biasanya digunakan untuk:

- menjalankan sesuatu **setelah** komponen muncul
- butuh respon terhadap perubahan props atau state
- efek hanya boleh dijalankan **satu kali**
- efek harus **dibersihkan** saat komponen hilang

Karena jika suatu kode kita letakkan tanpa `useEffect` maka ia akan terus dijalankan setiap komponen dirender.

Sebelum menerapkan `useEffect`, silahkan ikuti tahapan berikut:

📌 Buatlah sebuah halaman baru di Aplikasi Sedap dengan nama "Products"

▼ /data/product-sedap.json

```
[
  {
    "title": "Nasi Goreng Special",
    "category": "Makanan",
    "price": 25000,
    "image": "https://via.placeholder.com/100x100?text=Nasi+Goreng",
    "brand": "Warung Makan Pak Didi"
  },
  {
    "title": "Ayam Bakar Madu",
    "category": "Makanan",
    "price": 35000,
    "image": "https://via.placeholder.com/100x100?text=Ayam+Bakar",
    "brand": "Warung Makan Pak Didi"
  },
  {
    "title": "Es Teh Manis",
    "category": "Minuman",
    "price": 5000,
    "image": "https://via.placeholder.com/100x100?text=Es+Teh",
    "brand": "Minuman Segar"
  },
  {
    "title": "Jus Alpukat",
    "category": "Minuman",
    "price": 15000,
    "image": "https://via.placeholder.com/100x100?text=Jus+Alpukat"
  }
]
```

```

xt=Jus+Alpukat",
  "brand": "Minuman Segar"
},
{
  "title": "Bakso Urat Jumbo",
  "category": "Makanan",
  "price": 27000,
  "image": "https://via.placeholder.com/100x100?te
xt=Bakso",
  "brand": "Bakso Mas Kumis"
},
{
  "title": "Sate Ayam 10 Tusuk",
  "category": "Makanan",
  "price": 30000,
  "image": "https://via.placeholder.com/100x100?te
xt=Sate+Ayam",
  "brand": "Sate Ponorogo"
},
{
  "title": "Mie Goreng Pedas",
  "category": "Makanan",
  "price": 20000,
  "image": "https://via.placeholder.com/100x100?te
xt=Mie+Goreng",
  "brand": "Mie Enak 99"
},
{
  "title": "Mie Kuah Ayam",
  "category": "Makanan",
  "price": 19000,
  "image": "https://via.placeholder.com/100x100?te
xt=Mie+Kuah",
  "brand": "Mie Enak 99"
},
{

```

```
    "title": "Roti Bakar Coklat Keju",
    "category": "Makanan",
    "price": 18000,
    "image": "https://via.placeholder.com/100x100?text=Roti+Bakar",
    "brand": "Roti Bakar Bang Jun"
  },
  {
    "title": "Roti Bakar Strawberry",
    "category": "Makanan",
    "price": 17000,
    "image": "https://via.placeholder.com/100x100?text=Roti+Strawberry",
    "brand": "Roti Bakar Bang Jun"
  },
  {
    "title": "Cappuccino Dingin",
    "category": "Minuman",
    "price": 16000,
    "image": "https://via.placeholder.com/100x100?text=Cappuccino",
    "brand": "Kopi Tepi Jalan"
  },
  {
    "title": "Kopi Susu Gula Aren",
    "category": "Minuman",
    "price": 18000,
    "image": "https://via.placeholder.com/100x100?text=Kopi+Susu",
    "brand": "Kopi Tepi Jalan"
  },
  {
    "title": "Soto Ayam",
    "category": "Makanan",
    "price": 22000,
    "image": "https://via.placeholder.com/100x100?te
```

```

xt=Soto+Ayam",
  "brand": "Soto Lamongan Bu Rini"
},
{
  "title": "Keripik Singkong Pedas",
  "category": "Cemilan",
  "price": 10000,
  "image": "https://via.placeholder.com/100x100?text=Keripik",
  "brand": "Snack Lokal"
},
{
  "title": "Keripik Balado",
  "category": "Cemilan",
  "price": 12000,
  "image": "https://via.placeholder.com/100x100?text=Keripik+Balado",
  "brand": "Snack Lokal"
},
{
  "title": "Es Kelapa Muda",
  "category": "Minuman",
  "price": 14000,
  "image": "https://via.placeholder.com/100x100?text=Kelapa+Muda",
  "brand": "Minuman Segar"
},
{
  "title": "Es Cendol",
  "category": "Minuman",
  "price": 13000,
  "image": "https://via.placeholder.com/100x100?text=Cendol",
  "brand": "Cendol Daeng"
},
{

```

```
    "title": "Es Cincau",
    "category": "Minuman",
    "price": 12000,
    "image": "https://via.placeholder.com/100x100?text=Cincau",
    "brand": "Cendol Daeng"
  },
  {
    "title": "Pempek Palembang",
    "category": "Makanan",
    "price": 28000,
    "image": "https://via.placeholder.com/100x100?text=Pempek",
    "brand": "Pempek Bu Lina"
  },
  {
    "title": "Dimsum Ayam",
    "category": "Makanan",
    "price": 25000,
    "image": "https://via.placeholder.com/100x100?text=Dimsum",
    "brand": "Dimsum 88"
  },
  {
    "title": "Dimsum Udang",
    "category": "Makanan",
    "price": 27000,
    "image": "https://via.placeholder.com/100x100?text=Dimsum+Udang",
    "brand": "Dimsum 88"
  },
  {
    "title": "Boba Milk Tea",
    "category": "Minuman",
    "price": 22000,
    "image": "https://via.placeholder.com/100x100?te
```

```

xt=Boba",
    "brand": "Chatime"
  },
  {
    "title": "Boba Brown Sugar",
    "category": "Minuman",
    "price": 23000,
    "image": "https://via.placeholder.com/100x100?te
xt=Boba+Sugar",
    "brand": "Chatime"
  },
  {
    "title": "Kue Lapis Legit",
    "category": "Dessert",
    "price": 30000,
    "image": "https://via.placeholder.com/100x100?te
xt=Lapis+Legit",
    "brand": "Toko Kue Cita Rasa"
  },
  {
    "title": "Kue Cubit Coklat",
    "category": "Dessert",
    "price": 15000,
    "image": "https://via.placeholder.com/100x100?te
xt=Kue+Cubit",
    "brand": "Toko Kue Cita Rasa"
  },
  {
    "title": "Donat Coklat",
    "category": "Cemilan",
    "price": 10000,
    "image": "https://via.placeholder.com/100x100?te
xt=Donat",
    "brand": "Jco"
  },
  {

```

```

        "title": "Donat Matcha",
        "category": "Cemilan",
        "price": 11000,
        "image": "https://via.placeholder.com/100x100?text=Donat+Matcha",
        "brand": "Jco"
    },
    {
        "title": "Salad Buah",
        "category": "Dessert",
        "price": 18000,
        "image": "https://via.placeholder.com/100x100?text=Salad",
        "brand": "Fresh Bites"
    },
    {
        "title": "Pisang Goreng Keju",
        "category": "Cemilan",
        "price": 12000,
        "image": "https://via.placeholder.com/100x100?text=Pisang+Goreng",
        "brand": "Gorengan Maknyus"
    },
    {
        "title": "Tahu Gejrot",
        "category": "Cemilan",
        "price": 10000,
        "image": "https://via.placeholder.com/100x100?text=Tahu+Gejrot",
        "brand": "Gorengan Maknyus"
    }
]

```

▼ /pages/Products.jsx


```

import PageHeader from "../components/PageHeader"
import products from "../data/product-sedap.json"

export default function Products() {
  const breadcrumb = ["Dashboard", "Product List"]
  return (
    <div>
      <PageHeader title="Products" breadcrumb=
{breadcrumb} />

      <table className="min-w-full divide-y di
vide-gray-200 overflow-hidden rounded-2xl shadow-l
g">
        <thead>
          <tr className="bg-emerald-600 te
xt-white text-left text-sm font-semibold">
            <th className="px-4 py-3">#
</th>
            <th className="px-4 py-3">Na
me</th>
            <th className="px-4 py-3">Ca
tegory</th>
            <th className="px-4 py-3">Pr
ice</th>
            <th className="px-4 py-3">Ve
ndor</th>
          </tr>
        </thead>
        <tbody className="bg-white divide-y
divide-gray-100 text-sm text-gray-800">
          {products.map((item, index) => (
            <tr
              key={item.id}
              className="hover:bg-gray
-50 transition-colors duration-200"

```

```

        >
        <td className="px-6 py-4
font-medium text-gray-700">
            {index + 1}.
        </td>
        <td className="px-6 py-
4">{item.title}</td>
        <td className="px-6 py-
4">{item.category}</td>
        <td className="px-6 py-
4">Rp {item.price * 1000}</td>
        <td className="px-6 py-
4">{item.brand}</td>
    </tr>
    )})
</tbody>
</table>
</div>
)
}

```

▼ /components/Sidebar.jsx

```

<li>
    <NavLink
        id="menu-4"
        to="/products"
        className={menuClass}
    >
        <MdFastfood className="mr-4 text-xl" />
        Products
    </NavLink>
</li>

```

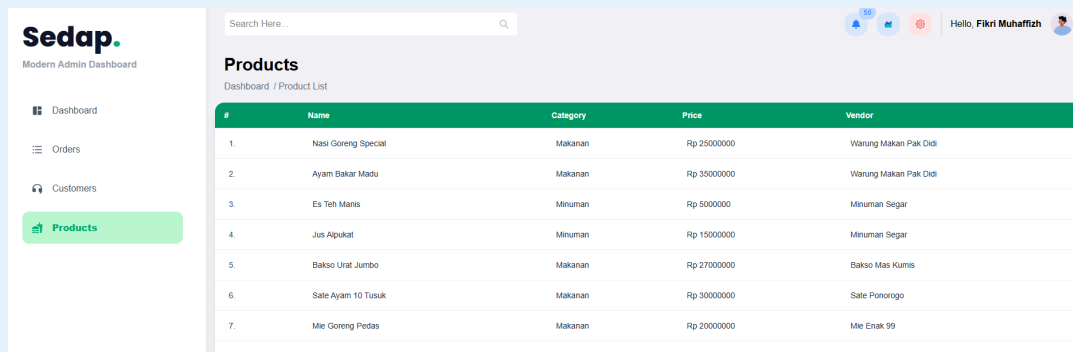
▼ App.jsx

```

<Route element={<MainLayout />}>
  ...
  <Route path="products" element={<Products /
>} />
</Route>

```

▼ Hasil Akhir



The screenshot shows a web application titled "Sedap. Modern Admin Dashboard". On the left is a sidebar with navigation links: Dashboard, Orders, Customers, and Products (highlighted in green). The main content area is titled "Products" and "Dashboard / Product List". It contains a table with the following data:

#	Name	Category	Price	Vendor
1.	Nasi Goreng Special	Makanan	Rp 25000000	Warung Makan Pak Didi
2.	Ayam Bakar Madu	Makanan	Rp 35000000	Warung Makan Pak Didi
3.	Es Teh Manis	Minuman	Rp 5000000	Minuman Segar
4.	Jus Alpukat	Minuman	Rp 15000000	Minuman Segar
5.	Bakso Urat Jumbo	Makanan	Rp 27000000	Bakso Mas Kumis
6.	Sate Ayam 10 Tusuk	Makanan	Rp 30000000	Sate Ponorogo
7.	Mie Goreng Pedas	Makanan	Rp 20000000	Mie Enak 99

Sekarang kita akan mengganti data produk yang awalnya diambil dari file JSON , menjadi data yang diambil langsung dari **API produk milik DummyJson**. Kita akan lakukan dengan dua skenario yang berbeda:

▼ Tanpa `useEffect`

Sesuaikan kode diatas dengan perubahan berikut:

▼ Definisikan state `products` , `error` dan `axios` untuk meng-handle pemanggilan API beserta informasi error. *(jangan lupa untuk menon-aktifkan data yang berasal dari json)*

```

import { BsFillExclamationDiamondFill } from "react-i
cons/bs";
// import products from "../data/product.json"
...

export default function Products() {

```

```

const [products, setProducts] = useState([])
const [error, setError] = useState(null)

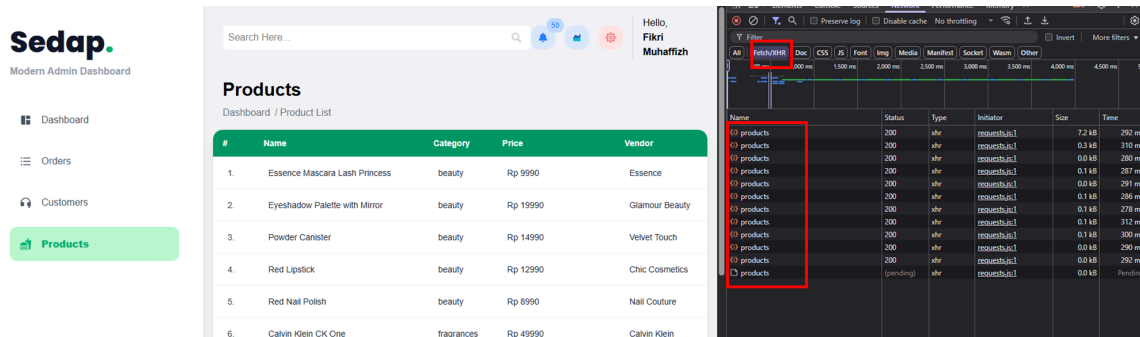
axios
  .get("https://dummyjson.com/products")
  .then((response) => {
    if (response.status !== 200) {
      setError(response.data.message);
      return;
    }
    setProducts(response.data.products)
  })
  .catch((err) => {
    setError(err.message || "An unknown error
occurred");
  });

const errorInfo = error ? (
  <div className="bg-red-200 mb-5 p-5 text-
sm font-light text-gray-600 rounded flex items-cente
r">
    <BsFillExclamationDiamondFill classNa
me="text-red-600 me-2 text-lg" />
    {error}
  </div>
) : null

return (
  ...
  {errorInfo}
  ...
)
}

```

Perubahan diatas berhasil melakukan pemanggilan data API, namun terjadi perluangan dalam melakukan request API karena axios langsung dipanggil tanpa menggunakan `useEffect`



▼ Penjelasan lebih detail kenapa terjadi request berulang

🧩 Langkah 1 – Komponen pertama kali dirender:

- React menjalankan `Products()`
- `useState([])` membuat `products` kosong
- `axios.get(...)` langsung dipanggil (karena bukan di dalam `useEffect`)
- Ketika `axios` selesai, dia memanggil `setProducts(...)`

🧩 Langkah 2 – State berubah:

- `setProducts()` membuat nilai `products` berubah dari `[]` menjadi data produk
- React mendeteksi perubahan state → maka komponen harus dirender ulang

🧩 Langkah 3 – Komponen dirender ulang:

- Karena adanya render ulang, React menjalankan ulang function `Products()`
- Di dalam function tersebut, `axios.get()` dipanggil lagi
- Lalu ketika response datang → `setProducts()` dipanggil lagi
- Ulangi terus... 🔄 infinite loop

▼ Dengan `useEffect`

Sekarang kita akan coba sesuaikan pemanggilan `axios` dengan menerapkan `useEffect`

Kita akan coba menampilkan list data product menggunakan DummyJson lalu menampilkannya ke database menggunakan `useEffect`. Lakukan perubahan berikut:

Before

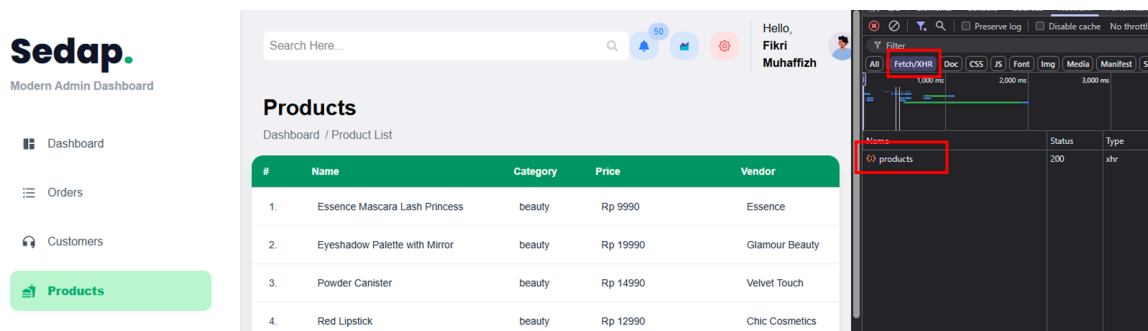
```
axios
  .get("https://dummyjson.com/products")
  .then((response) => {
    if (response.status !== 200) {
      setError(response.data.message)
      return
    }
    setProducts(response.data.products)
  })
  .catch((err) => {
    setError(err.message || "An unknown error occurred")
  })
```

After

```
useEffect(() => {
  axios
    .get("https://dummyjson.com/products")
    .then((response) => {
      if (response.status !== 200) {
        setError(response.data.message)
        return
      }
      setProducts(response.data.products)
    })
    .catch((err) => {
      setError(err.message || "An unknown error occurred")
    })
})
```

```
}, []) // <- array kosong  
artinya hanya dijalankan  
sekali setelah di  
-render
```

Setelah `useEffect` diterapkan, `axios` hanya dipanggil sekali setelah render dilakukan



Terdapat 3 jenis `useEffect` yang bisa kita terapkan:

```
useEffect(() => { ... }, [ dependencies ]);
```

1. Tanpa Dependency

```
useEffect(() => {  
  console.log("Ini akan dijalankan setiap kali komponen dirender");  
});
```

2. Dengan Dependency Array Kosong `[]` (ini diterapkan pada contoh sebelumnya)

```
useEffect(() => {  
  console.log("Hanya dijalankan 1x saat komponen pertama kali render");  
}, []);
```

```
ama kali dimuat");  
}, []);
```

3. Dengan Dependency Tertentu `[state]`

```
useEffect(() => {  
    console.log("Kode ini akan jalan setiap kali nilai  
`error` berubah");  
}, [error]);  
  
useEffect(() => {  
    console.log("Kode ini akan jalan setiap kali nilai  
`a` atau nilai `b` berubah");  
}, [a, b]);
```

▼ `useEffect` dengan dependencies state

Sebelumnya, kita telah mencoba menggunakan `useEffect` dengan dependency array kosong `[]`, yang berarti proses di dalamnya hanya akan dijalankan sekali saat komponen pertama kali dirender.

Sekarang, kita akan mencoba menggunakan `useEffect` dengan dependency berupa nilai dari sebuah `state`. Artinya, proses di dalam `useEffect` akan dijalankan setiap kali nilai `state` tersebut mengalami perubahan.

```
useEffect(() => { ... }, [ dependencies ]);
```

Kita akan menambah fitur search berdasarkan data product. Lakukan perubahan berikut:

- ▼ **1** Buat state baru yaitu `query`


```
const [query, setQuery] = useState("")
```

- ▼ **2** Ubah `useEffect` dan `axios` agar mengambil data berdasarkan state

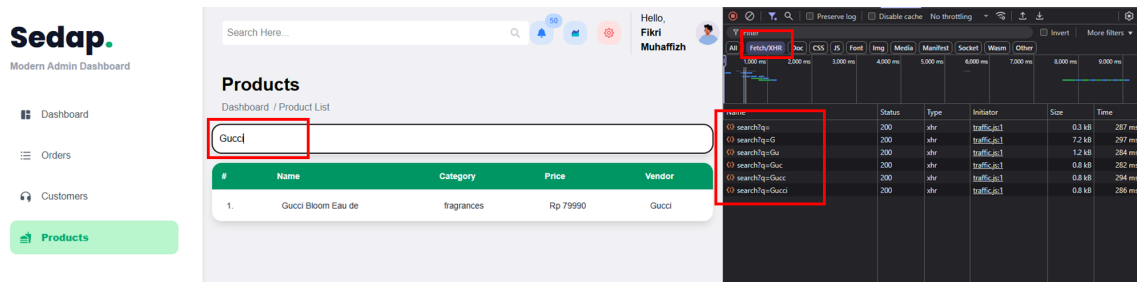
`query`

```
useEffect(() => {  
  axios  
    .get(`https://dummyjson.com/products/search?q=${query}`) // menerapkan search dan query param  
    .then((response) => {  
      ...  
    })  
    .catch((err) => {  
      ...  
    })  
}, [query]) // -> useEffect akan dipanggil ulang setiap  
`query` berubah
```

- ▼ **3** Tambahkan kolom search untuk mencari berdasarkan nama produk
(*posisikan diatas tabel*)

```
<input  
  type="text"  
  value={query}  
  onChange={(e) => setQuery(e.target.value)}  
  placeholder="Cari produk..."  
  className="mb-4 p-3 w-full bg-white rounded-2xl shadow-lg"  
>  
  
<table>  
  ...  
</table>
```

- ▼ 4 API akan dipanggil setiap state `query` berubah

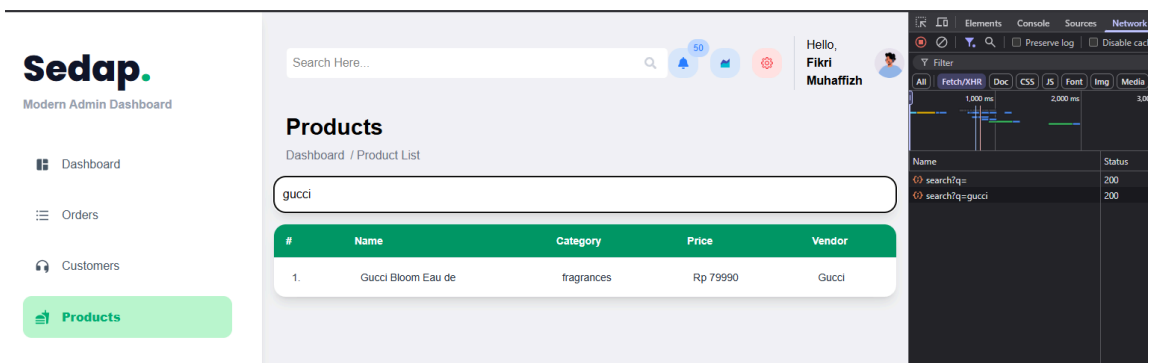


Fitur search diatas sudah berhasil dilakukan, namun kita bisa melakukan optimasi lebih baik agar API tidak dipanggil setiap ketikan, namun dijalankan 500ms setelah user selesai mengetik (debounce). lakukan perubahan berikut

- ▼ 5 Terapkan debounce menggunakan `timeout`

```
useEffect(() => {
  const timeout = setTimeout(() => {
    axios.get(`https://dummyjson.com/products/search?q=${query}`)
    ...
  }, 500); // 500ms debounce

  return () => clearTimeout(timeout); // cleanup
}, [query]);
```



Setelah user selesai mengetikkan "gucci", barulah API di panggil sehingga server tidak terbebani dengan request yang tidak perlu

Contoh diatas mirip dengan `handleChange` yang kita lakukan saat Pertemuan 4, namun kode tersebut tidak sebaik menggunakan `useEffect` karena:

- `onChange + axios langsung` : semua logic pencarian langsung di handler, tidak bisa dikontrol dan tidak rapi.
- `onChange -> setState -> useEffect axios` : terdapat tempat khusus (yaitu `useEffect`) untuk melakukan side effect (seperti axios), yang:
 - lebih **terstruktur**,
 - bisa **ditambahkan debounce**, cancel, validasi, dll dengan lebih fleksibel.

▼ Dynamic Route

Dynamic Route adalah route yang memungkinkan URL untuk menerima nilai dinamis yang bisa berubah. Biasanya digunakan untuk halaman detail, contoh:

▼ `/customers/1` → detail produk dengan id 1

Dalam React Router, kita menuliskannya seperti ini:

```
<Route path="/customers/:id" element={<CustomerDetail />} />
```

lalu kita bisa ambil nilainya dengan cara berikut:

```
const { id } = useParams()
```

▼ `/users/abc123` → detail user dengan ID `abc123`

Dalam React Router, kita menuliskannya seperti ini:

```
<Route path="/users/:abc" element={<UserDetail />} />
```

lalu kita bisa ambil nilainya dengan cara berikut:

```
const { abc } = useParams()
```

Kita akan coba membuat halaman detail dari list product berdasarkan ID. Silahkan ikut tahapan berikut:

- ▼ **1** Buat route baru di `App.jsx` serta import dengan `React.Lazy()`

```
...  
const ProductDetail = React.lazy(() => import("./pages/  
ProductDetail"))  
...  
  
<Route element={<MainLayout />}>  
  <Routes>  
    ...  
    <Route path="/products/:id" element={<P  
roductDetail />} />  
  </Routes>  
</Route>
```

- ▼ **2** Ubah nama produk menjadi link ke detail

```
<td className="px-6 py-4">  
  <Link to={`/${products/${item.id}}`} className="text-e  
merald-400 hover:text-emerald-500">  
    {item.title}  
  </Link>  
</td>
```

⚠ Jangan lupa import komponen [Link](#)

- ▼ **3** Buat komponen baru: `ProductDetail.jsx` di folder Pages

```
import { useParams } from "react-router-dom"
import { useEffect, useState } from "react"
import axios from "axios"

export default function ProductDetail() {
  const { id } = useParams()
  const [product, setProduct] = useState(null)
  const [error, setError] = useState(null)

  useEffect(() => {
    axios
      .get(`https://dummyjson.com/products/${id}`)
      .then((response) => {
        if (response.status !== 200) {
          setError(response.message)
          return
        }
        setProduct(response.data)
      })
      .catch((err) => {
        setError(err.message)
      })
  }, [id])

  if (error) return <div className="text-red-600 p-4">{error}</div>
  if (!product) return <div className="p-4">Loading...</div>
```

```

    return (
      <div className="p-6 bg-white rounded-xl shadow-
lg max-w-lg mx-auto mt-6">
        <img
          src={product.thumbnail}
          alt={product.title}
          className="rounded-xl mb-4 w-full h-48
object-cover"
        />
        <h2 className="text-2xl font-bold mb-2">{pr
oduct.title}</h2>
        <p className="text-gray-600 mb-1">Kategori:
{product.category}</p>
        <p className="text-gray-600 mb-1">Brand: {p
roduct.brand}</p>
        <p className="text-gray-800 font-semibold t
ext-lg">
          Harga: Rp {product.price * 1000}
        </p>
      </div>
    )
  }

```

▼ 4 Hasil Akhir

Products
Dashboard / Product List

Cari produk...

#	Name	Category
1.	Essence Mascara Lash Princess	beauty
2.	Eyeshadow Palette with Mirror	beauty
3.	Powder Canister	beauty
4.	Red Lipstick	beauty

